1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.
a. Load dataset using Pandas.
b. Perform basic preprocessing (handling missing values or scaling).
c. Split dataset into training and testing sets.
d. Train a simple classifier.
e. Display training and testing accuracy.

Machine Learning Workflow using Iris Dataset

```
# Step 1: Import required libraries
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Step 2: Load dataset using Pandas
iris = load_iris()

# Create a Pandas DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

print("First 5 rows of the dataset:")
print(df.head())

# Step 3: Basic Preprocessing
# Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Features and Target
```

```
X = df.iloc[:, :-1]
y = df['target']
# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Step 4: Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
# Step 5: Train a simple classifier
classifier = DecisionTreeClassifier(random_state=42)
classifier.fit(X_train, y_train)
# Step 6: Make predictions
train_pred = classifier.predict(X_train)
test_pred = classifier.predict(X_test)
# Step 7: Display training and testing accuracy
train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)
print("\nTraining Accuracy:", train_accuracy)
print("Testing Accuracy:", test_accuracy)
```

2. Using a suitable regression dataset, implement Linear Regression using scikit-learn.

```
# Linear Regression using Scikit-learn
# Step 1: Import required libraries
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Step 2: Load the Diabetes dataset
diabetes = load_diabetes()

# Convert dataset into a Pandas DataFrame
df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
df['target'] = diabetes.target

# Display first five rows
print("First 5 rows of the dataset:")
print(df.head())

# Step 3: Define Features (X) and Target (y)
X = df.drop('target', axis=1)
y = df['target']

# Step 4: Split the dataset into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Create the Linear Regression model
model = LinearRegression()
```

```
# Step 6: Train the model
model.fit(X_train, y_train)

# Step 7: Predict the test data
y_pred = model.predict(X_test)

# Step 8: Display coefficients and intercept
print("\nIntercept:", model.intercept_)
print("Coefficients:", model.coef_)

# Step 9: Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nMean Squared Error:", mse)
print("R² Score:", r2)
```

3. Using the MNIST dataset (digit vs not digit), implement a binary classifier.

```
# Binary Classification using MNIST Dataset

# Step 1: Import required libraries
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load the MNIST dataset
mnist = fetch_openml('mnist_784', version=1, as_frame=False)

# Features and Target
X = mnist.data
y = mnist.target.astype(int)

# Step 3: Convert the problem into Binary Classification
# True if digit is 5, otherwise False
y_binary = (y == 5)

# Step 4: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y_binary, test_size=0.2, random_state=42
)

# Step 5: Create the Binary Classifier
classifier = SGDClassifier(random_state=42)

# Step 6: Train the classifier
classifier.fit(X_train, y_train)
```

```
# Step 7: Predict the test data
y_pred = classifier.predict(X_test)

# Step 8: Display Results
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

4. Implement k-fold cross-validation on a classification model.

```
# K-Fold Cross Validation using Iris Dataset

# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import KFold, cross_val_score
from sklearn.tree import DecisionTreeClassifier

# Step 2: Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Step 3: Create the classifier
classifier = DecisionTreeClassifier(random_state=42)

# Step 4: Define K-Fold Cross Validation
kfold = KFold(n_splits=5, shuffle=True, random_state=42)

# Step 5: Perform Cross Validation
scores = cross_val_score(classifier, X, y, cv=kfold)

# Step 6: Display Results
print("Accuracy for each fold:")
print(scores)

print("\nAverage Accuracy:", scores.mean())

print("Standard Deviation:", scores.std())
```

5. Implement a Decision Tree classifier using the Iris or any relevant dataset.

```
# Decision Tree Classifier using Iris Dataset

# Step 1: Import required libraries
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

# Convert to Pandas DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Display first five rows
print("First 5 rows of the dataset:")
print(df.head())

# Step 3: Define Features and Target
X = df.iloc[:, :-1]
y = df['target']

# Step 4: Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Step 5: Create the Decision Tree Classifier
classifier = DecisionTreeClassifier(random_state=42)

# Step 6: Train the model
classifier.fit(X_train, y_train)

# Step 7: Predict the test data
y_pred = classifier.predict(X_test)

# Step 8: Evaluate the model
print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

6. Implement Decision Tree regression on a regression dataset.

```
# Decision Tree Regression using Diabetes Dataset

# Step 1: Import required libraries
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Step 2: Load the Diabetes dataset
diabetes = load_diabetes()

# Convert dataset into a Pandas DataFrame
df = pd.DataFrame(diabetes.data, columns=diabetes.feature_names)
df['target'] = diabetes.target

# Display first five rows
print("First 5 rows of the dataset:")
print(df.head())

# Step 3: Define Features (X) and Target (y)
X = df.drop('target', axis=1)
y = df['target']

# Step 4: Split the dataset into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Create the Decision Tree Regressor
```

```
model = DecisionTreeRegressor(random_state=42)

# Step 6: Train the model
model.fit(X_train, y_train)

# Step 7: Predict the test data
y_pred = model.predict(X_test)

# Step 8: Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\nMean Squared Error (MSE):", mse)
print("R² Score:", r2)
```

7. Implement k-Nearest Neighbors (kNN) classifier.

k-Nearest Neighbors (kNN) Classifier using Iris Dataset

```
# Step 1: Import required libraries
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

# Convert dataset into a Pandas DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Display first five rows
print("First 5 rows of the dataset:")
print(df.head())

# Step 3: Define Features (X) and Target (y)
X = df.iloc[:, :-1]
y = df['target']

# Step 4: Split the dataset into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Create the kNN Classifier
```

```
knn = KNeighborsClassifier(n_neighbors=3)

# Step 6: Train the model
knn.fit(X_train, y_train)

# Step 7: Predict the test data
y_pred = knn.predict(X_test)

# Step 8: Evaluate the model
print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

8. Implement multiclass classification using Logistic Regression or Decision Tree.

```
# Multiclass Classification using Logistic Regression

# Step 1: Import required libraries
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

# Convert dataset into a Pandas DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Display first five rows
print("First 5 rows of the dataset:")
print(df.head())

# Step 3: Define Features (X) and Target (y)
X = df.iloc[:, :-1]
y = df['target']

# Step 4: Split the dataset into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Create the Logistic Regression model
```

```
model = LogisticRegression(max_iter=200)

# Step 6: Train the model
model.fit(X_train, y_train)

# Step 7: Predict the test data
y_pred = model.predict(X_test)

# Step 8: Evaluate the model
print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Module 2: Jira Setup & Project Configuration, Advanced Configuration, Reporting & Governance

---

Practical 1: Jira Setup & Project Creation

A. Create a Jira Cloud Account and Explore Dashboard

Steps:

1. Open the Jira Cloud website.
2. Create a Jira account using the required email address.
3. Log in to Jira Cloud.
4. Explore the Jira dashboard and navigation menu.
5. Check the available projects, boards, issues, and other Jira features.

B. Create a Company-managed Scrum/Kanban Project

Steps:

1. Open **Projects** from the Jira navigation menu.
2. Select **Create Project**.
3. Select the **Software Development** category.
4. Choose a **Scrum** or **Kanban** template.
5. Select the **Company-managed** project type.
6. Enter the project name and project key.
7. Click **Create Project**.

C. Configure Project Details

Steps:

1. Open the created project.
2. Go to **Project/Space Settings**.
3. Open the project details section.
4. Configure the project name, description, and other available settings.
5. Save the changes.

Result: A Jira Cloud project was successfully created and configured.

Practical 2: Team Creation & Role Assignment

A. Add Team Members

Steps:

1. Open the Jira project.
2. Navigate to **Project/Space Settings → People**.
3. Click **Add people**.
4. Search for the required Jira users.
5. Select the users to be added.
6. Add them to the project.

B. Assign Roles

Steps:

1. Open the **People** section of the project.
2. Locate the required user.
3. Open the **Role** dropdown.
4. Select the appropriate role such as **Administrator** or **COPO Developers**.
5. Save the role assignment.
6. Verify that the user's assigned role is displayed.

Result: Team members were added and appropriate project roles were assigned.

Practical 3: Issue Types & Configuration

A. Create and Configure Issue Types

Steps:

1. Open the Jira project.
2. Navigate to the issue creation screen.
3. Select the required issue type.
4. Create issues using **Epic, Story, Task, and Bug** types.
5. Enter the issue summary and required details.
6. Save each issue.

B. Create Components and Versions

Steps:

1. Open **Project/Space Settings**.
2. Navigate to **Components**.
3. Click **Create Component**.
4. Enter the component name and required details.

5. Save the component.
6. Open **Versions/Releases**.
7. Create a new version.
8. Enter the version name and required details.
9. Save the version.

Result: Different Jira issue types, components, and project versions were successfully created.

Practical 4: Issue Management

A. Create Issues Under an Epic

Steps:

1. Open the Jira project.
2. Create an **Epic**.
3. Enter the Epic name and details.
4. Create Story, Task, or Bug issues.
5. Link the issues to the appropriate Epic using the parent/epic relationship.
6. Verify that the issues appear under the Epic.

B. Assign Issues and Set Priority & Due Date

Steps:

1. Open the required issue.
2. Select an appropriate **Assignee**.
3. Set the issue **Priority**.
4. Set the **Due Date**.
5. Save or update the issue.
6. Verify the assigned user, priority, and due date.

Result: Issues were created under an Epic and configured with appropriate assignment, priority, and due dates.

Practical 5: Workflow Customization

A. Design a Custom Workflow

Steps:

1. Open the Jira project settings.

2. Navigate to the workflow configuration section.
3. Create a new custom workflow.
4. Add required statuses such as **To Do, In Progress, Testing, and Done**.
5. Arrange the statuses according to the required workflow.

B. Add Transitions and Conditions

Steps:

1. Create transitions between the workflow statuses.
2. Configure transitions such as **To Do → In Progress**.
3. Add transitions between **In Progress → Testing** and **Testing → Done**.
4. Configure required conditions for the transitions.
5. Save the workflow configuration.

C. Publish and Associate Workflow

Steps:

1. Review the completed workflow.
2. Publish the workflow.
3. Associate the workflow with the required Jira project.
4. Verify that the project uses the custom workflow.
5. Test the workflow by moving an issue through its statuses.

Result: A custom workflow was created, configured, published, and associated with the project.

Practical 6: Scrum Board & Sprint Management

A. Create and Configure Scrum Board

Steps:

1. Open the Jira project.
2. Navigate to **Boards**.
3. Create a new **Scrum Board**.
4. Select the appropriate project or project filter.
5. Configure the board columns according to the workflow.

B. Create and Start a Sprint

Steps:

1. Open the Scrum Board backlog.

2. Create a new Sprint.
3. Add the required issues to the Sprint.
4. Set the Sprint name and duration.
5. Click **Start Sprint**.
6. Verify that the Sprint is active.

C. Complete Sprint and Generate Sprint Report

Steps:

1. Open the active Sprint.
2. Complete the required issues.
3. Move issues through the workflow.
4. Select **Complete Sprint** after the Sprint work is finished.
5. Open the **Reports** section.
6. Select **Sprint Report**.
7. View the completed and incomplete issues.

Result: A Scrum Board and Sprint were created, managed, completed, and analyzed using the Sprint Report.

Practical 7: Kanban Board Management

A. Create a Company-managed Kanban Project

Steps:

1. Open **Projects → Create Project**.
2. Select the Software/Kanban template.
3. Choose **Company-managed**.
4. Enter the project name and project key.
5. Create the project.

B. Configure Kanban Board

Steps:

1. Open the Kanban Board.
2. Navigate to the board configuration.
3. Configure the required columns.
4. Map workflow statuses to the appropriate columns.

5. Configure other board settings if required.
6. Save the configuration.

C. Track Issue Progress and Generate Control Chart

Steps:

1. Create and add issues to the Kanban Board.
2. Move issues between board columns as work progresses.
3. Monitor the issue status on the board.
4. Open the **Reports** section.
5. Select **Control Chart**.
6. View the time taken by issues through the workflow.

Result: A Kanban Board was created and configured, and issue progress was analyzed using the Control Chart.

Practical 8: JQL & Filters

A. Write Basic JQL Queries

Steps:

1. Open **Issues/Search for issues**.
2. Switch to **Advanced/JQL** search.
3. Enter a JQL query based on issue status.
4. Execute the query.
5. Create queries based on **Assignee** and **Priority**.
6. Verify the returned issues.

Examples:

project = COPO AND status = "Done"

project = COPO AND assignee = currentUser()

project = COPO AND priority = High

B. Create and Save Filters

Steps:

1. Enter the required JQL query.
2. Execute the search.
3. Click **Save as**.

4. Enter a filter name.
5. Save the filter.
6. Open the saved filter to verify the results.

Result: JQL queries were created and saved as reusable Jira filters.

Practical 9: Reports & Dashboards

A. Generate Burndown and Velocity Charts

Steps:

1. Open the Jira project.
2. Navigate to **Reports**.
3. Select **Burndown Chart**.
4. Select the required Sprint.
5. View the remaining Story Points over time.
6. Open the **Velocity Report**.
7. View Sprint commitment and completed Story Points.

B. Create Pie Chart / Issue Statistics Report

Steps:

1. Open the Jira dashboard.
2. Click **Add gadget**.
3. Search for **Pie Chart**.
4. Add the Pie Chart gadget.
5. Select the required project/filter and configure the statistic type.
6. Add the **Issue Statistics** gadget.
7. Configure it using the required project/filter and statistic type.
8. Save the gadget configuration.

C. Create a Custom Dashboard with Gadgets

Steps:

1. Open **Dashboards**.
2. Create a new dashboard.
3. Enter the dashboard name.
4. Add required gadgets.

5. Configure the gadgets using the project or saved filters.
6. Arrange the gadgets into the required layout.
7. Save the dashboard.

Result: Jira reports were generated and a custom dashboard containing multiple gadgets was successfully created.

Practical 10: Permission & Security Management

A. View and Modify Project Permission Scheme

Steps:

1. Open the Jira project.
2. Navigate to **Project Settings → Permissions**.
3. View the existing **Permission Scheme**.
4. Review permissions such as Browse Projects, Manage Sprints, and Manage Versions.
5. Select the required permission.
6. Modify the users, groups, or project roles associated with the permission.
7. Save/update the permission.

B. Configure Project Roles and Assign Permissions

Steps:

1. Open the Jira administration/settings area.
2. Open **Project/Space Roles**.
3. View the available project roles.
4. Create or select the required project role.
5. Assign users to the appropriate role.
6. Return to the Permission Scheme.
7. Assign required permissions to the project role.
8. Verify the user's role and associated permissions.

Result: Project permissions and roles were configured to control user access and project activities.

Practical 11: Jira Automation

A. Create an Automation Rule

Steps:

1. Open **Project/Space Settings → Automation**.
2. Select **Create flow/Rule**.
3. Create a rule from scratch or use a template.
4. Configure the **Trigger**.
5. Add the required **Condition**, if needed.
6. Configure the **Action**.
7. Save the automation rule.

B. Configure the Automation Rule

Example: When an issue is moved to Done, automatically assign it to a user.

Steps:

1. Select **Work item transitioned** as the trigger.
2. Set the destination status as **Done**.
3. Add an **Assign work item** action.
4. Select the required user.
5. Review the complete automation flow.
6. Save the rule.

C. Test and Enable Automation

Steps:

1. Enable the automation rule.
2. Move a test issue to **Done**.
3. Check whether the configured action is performed.
4. Open the **Automation Audit Log**.
5. Verify that the rule execution shows **Success**.

Result: An automation rule was created, configured, enabled, tested, and successfully executed.

Practical 12: Introduction to Jira Mobile App

A. Install and Configure Jira Mobile App

Steps:

1. Install the **Jira mobile application** on a smartphone.
2. Open the application.
3. Sign in using the Jira account.

4. Select the required Jira project/space.
5. Open the project dashboard and verify the available options.

B. View and Update Issues from Mobile

Steps:

1. Open the Jira project from the mobile application.
2. Select an issue.
3. View the issue details.
4. Change the issue status.
5. Add a comment to the issue.
6. Assign the issue to a user.
7. Verify that the changes are updated in Jira.

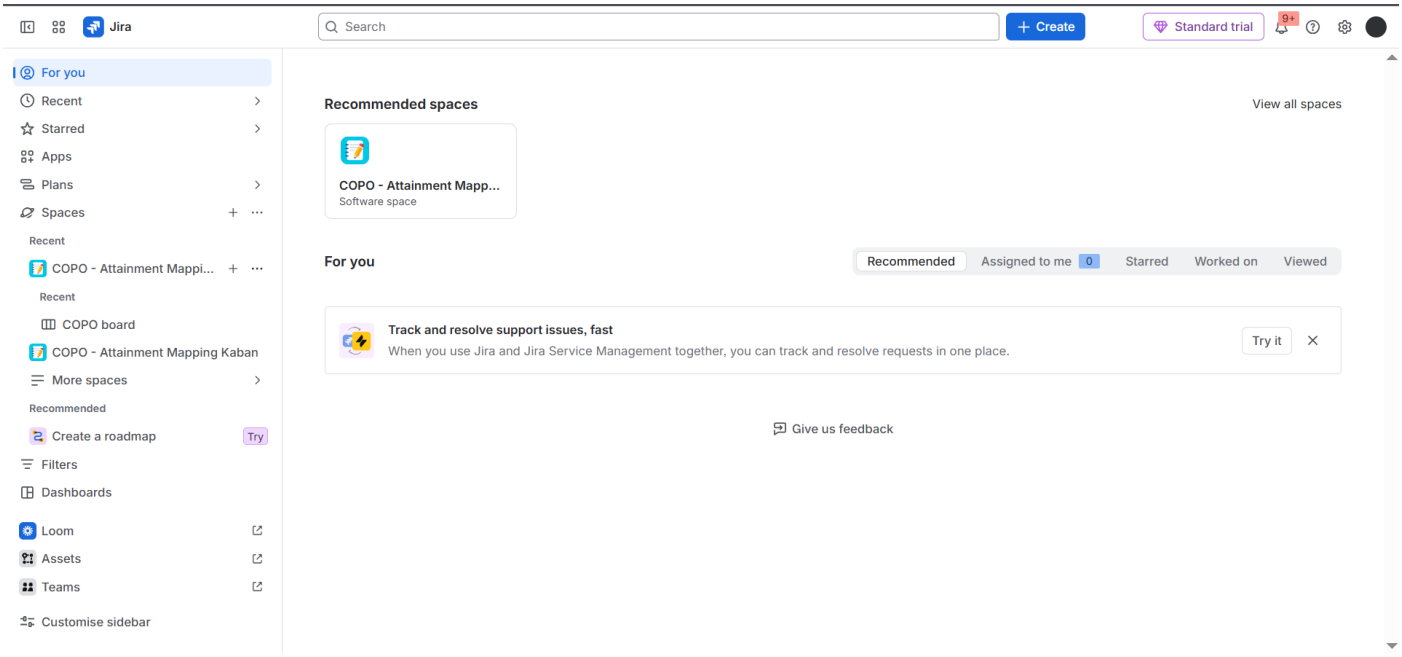
C. Receive Notifications and Manage Real-time Updates

Steps:

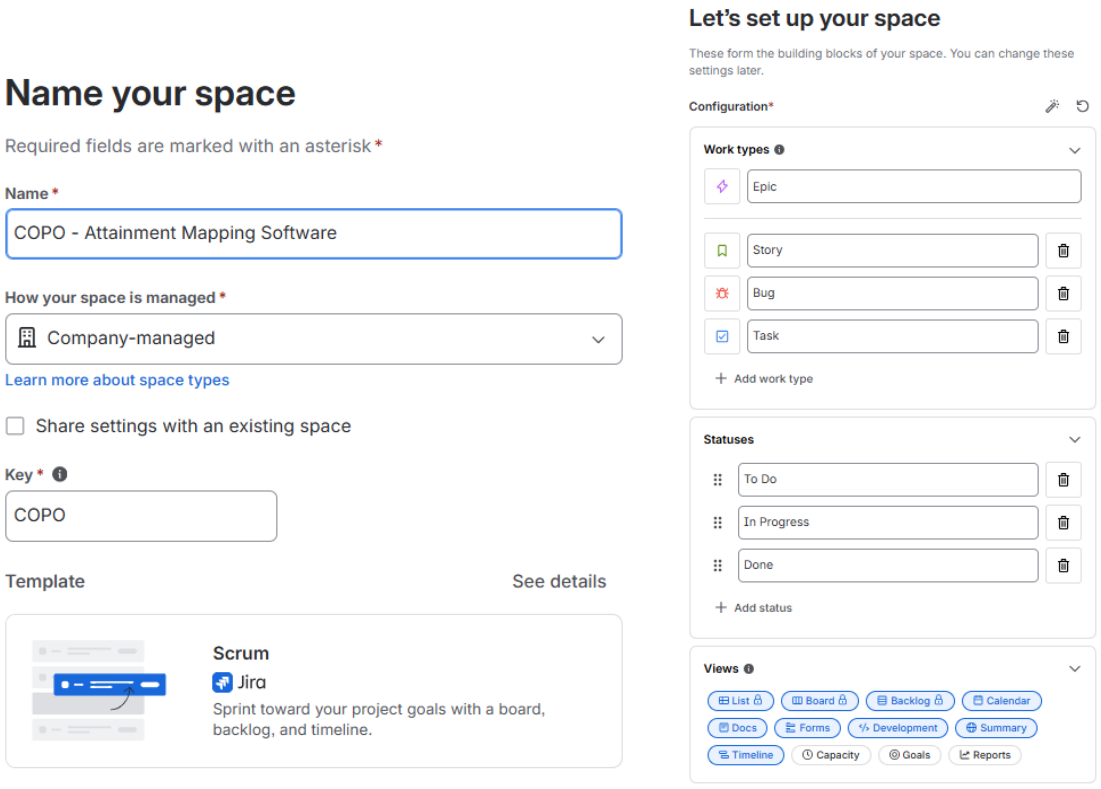
1. Enable Jira notifications on the mobile device.
2. Perform an issue update such as assignment or status change.
3. Check the mobile notification.
4. Open the notification to view the updated issue.
5. Verify that the latest issue information is displayed.

Result: Jira was successfully accessed through the mobile application, issues were updated, and real-time notifications were received.

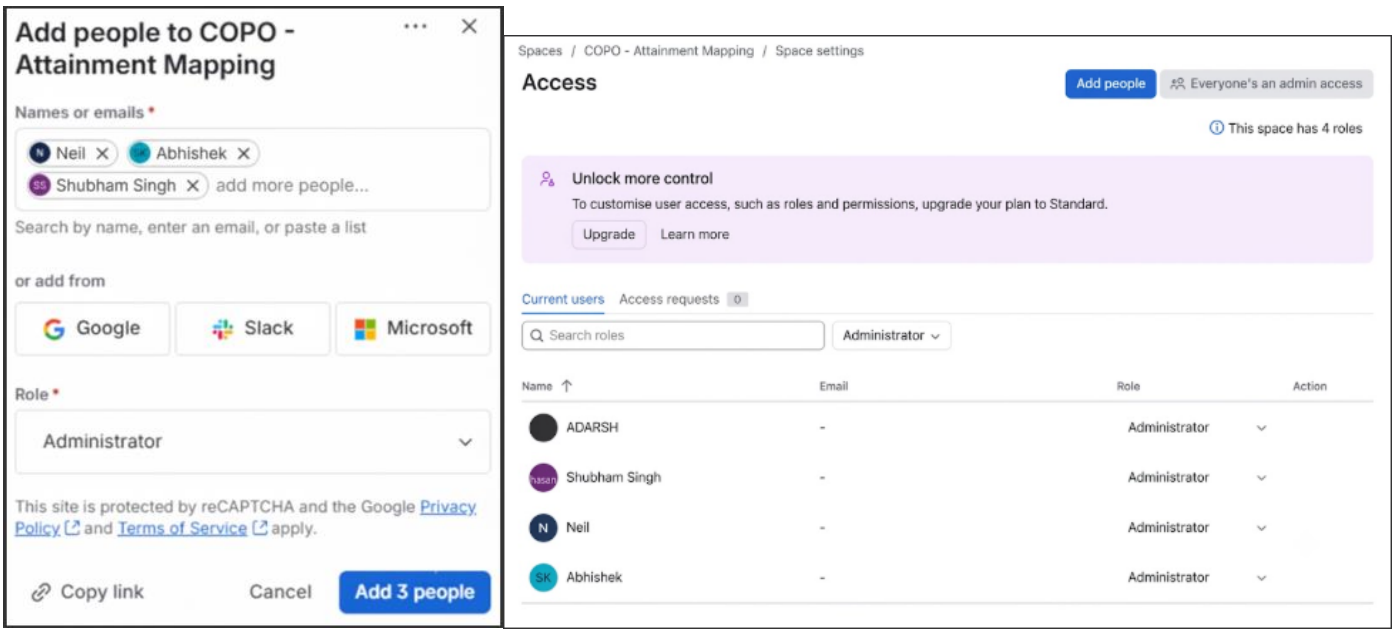
1A


Jira
Search
Create
Standard trial
For you
Recent
Starred
Apps
Plans
Spaces
Recent
COPO - Attainment Mappi...
Recent
COPO board
COPO - Attainment Mapping Kaban
More spaces
Recommended
Create a roadmap
Try
Filters
Dashboards
Loom
Assets
Teams
Customise sidebar
Recommended spaces
View all spaces
COPO - Attainment Mapp...
Software space
For you
Recommended
Assigned to me 0
Starred
Worked on
Viewed
Track and resolve support issues, fast
When you use Jira and Jira Service Management together, you can track and resolve requests in one place.
Try it
Give us feedback

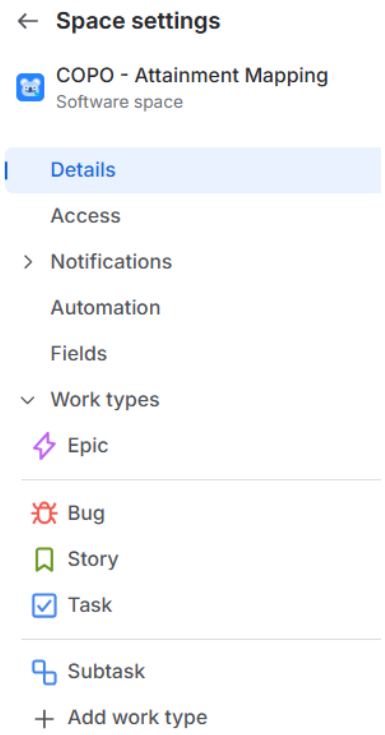
1B, 1C


Name your space
Required fields are marked with an asterisk *
Name *
COPO - Attainment Mapping Software
How your space is managed *
Company-managed
Learn more about space types
Share settings with an existing space
Key *
COPO
Template
See details
Scrum
Jira
Sprint toward your project goals with a board, backlog, and timeline.
Let's set up your space
These form the building blocks of your space. You can change these settings later.
Configuration*
Work types
Epic
Story
Bug
Task
Add work type
Statuses
To Do
In Progress
Done
Add status
Views
List
Board
Backlog
Calendar
Docs
Forms
Development
Summary
Timeline
Capacity
Goals
Reports

2A, 2B



3A



3B (1 i.e components & 2 i.e versions)

Component	Description	Component lead	Default assignee	Issues	More actions
Frontend	UI of the Software		Project default	0 issues	···
Documentation	Documents regarding project		Project default	0 issues	···
Database	Storing of COs and POs		Project default	0 issues	···
Backend	Copo processing logic		Project default	0 issues	···

Spaces / COPO - Attainment Mapping Software / Space settings

Releases

This space has 0 releases

Unreleased

Give feedback | Create release

Releases

Release	Status	Progress	Start date	Release date	Description	More actions
v2.0.0	RELEASED	No work items	9 September 2026	23 September 2026	Phase 1	...
v1.0.0	RELEASED	No work items	1 September 2026	8 September 2026	Prototype	...

4A

COPO-1

Create UI of Homescreen

4B

Work		Priority	Stor...	Assignee	Status	Due date
COPO-2	Create Login page	Highest		Neil	In Progress	30 Sept 2026

5A

Spaces / COPO - Attainment Mapping Software / Project settings

Workflows

COPO: Software Simplified Workflow Scheme

Add Workflow | Switch Scheme

Workflow	Work Types	Actions
Copy of COPO Project Development Workflow (View as text / diagram)	Epic Bug Story Task Sub-task	Edit workflow

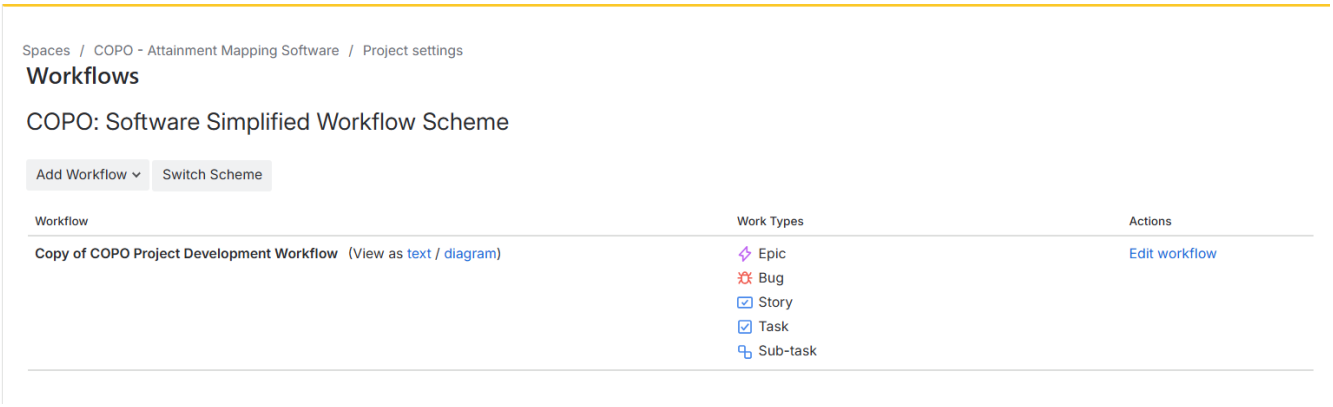
5B

Diagram | Text

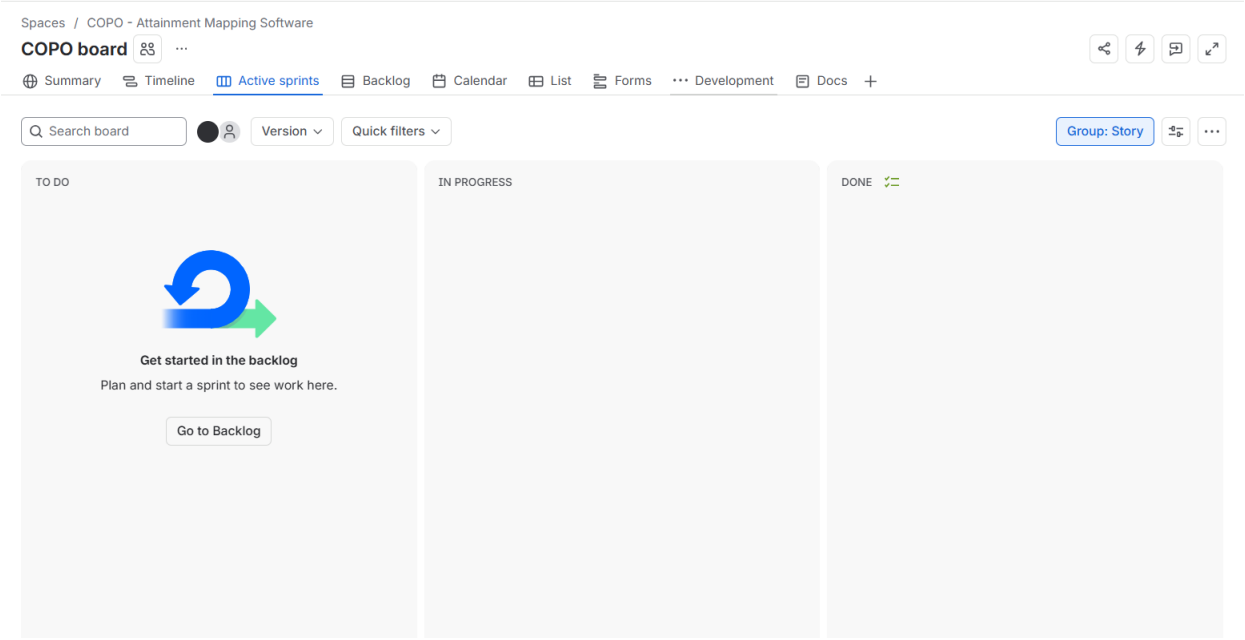
Status (ID)	Transitions (ID)
To Do (10043)	**Create** (1) Start → To Do
	From To Do to In Progress (2) To Do → In Progress
In Progress (3)	**From To Do to In Progress** (2) To Do → In Progress
	From In Progress to Testing (3) In Progress → Testing
Testing (10045)	**From In Progress to Testing** (3) In Progress → Testing
Done (10044)	**Done** (4) Any Status → Done

Give feedback

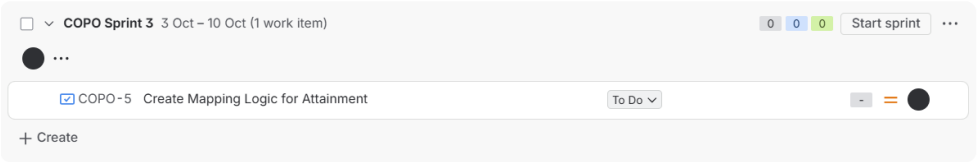
5C


Spaces / COPO - Attainment Mapping Software / Project settings
Workflows
COPO: Software Simplified Workflow Scheme
Add Workflow
Switch Scheme
Workflow
Work Types
Actions
Copy of COPO Project Development Workflow (View as text / diagram)
Epic
Bug
Story
Task
Sub-task
Edit workflow

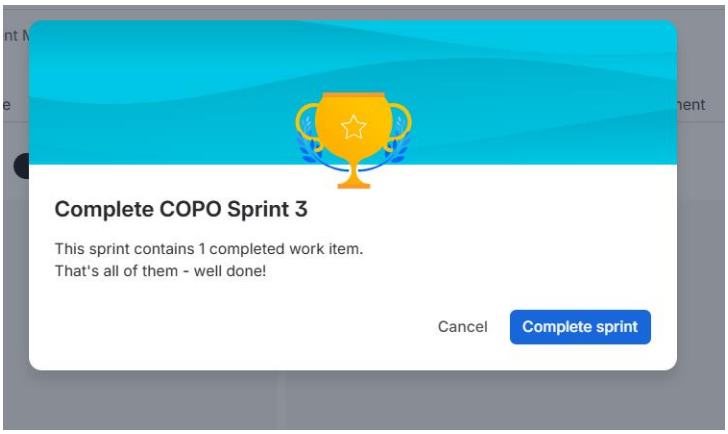
6A


Spaces / COPO - Attainment Mapping Software
COPO board
Summary
Timeline
Active sprints
Backlog
Calendar
List
Forms
Development
Docs
Search board
Version
Quick filters
Group: Story
TO DO
IN PROGRESS
DONE
Get started in the backlog
Plan and start a sprint to see work here.
Go to Backlog

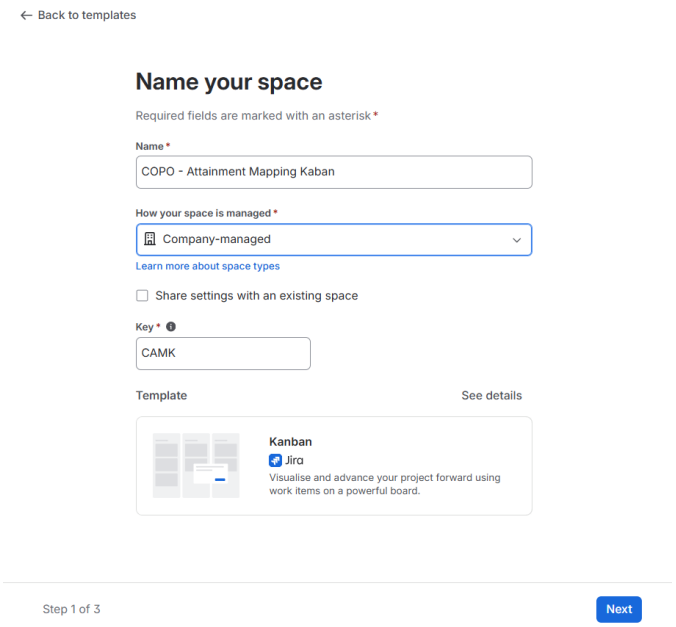
6B:


COPO Sprint 3 3 Oct – 10 Oct (1 work item)
0 0 0
Start sprint
COPO-5 Create Mapping Logic for Attainment
To Do
Create

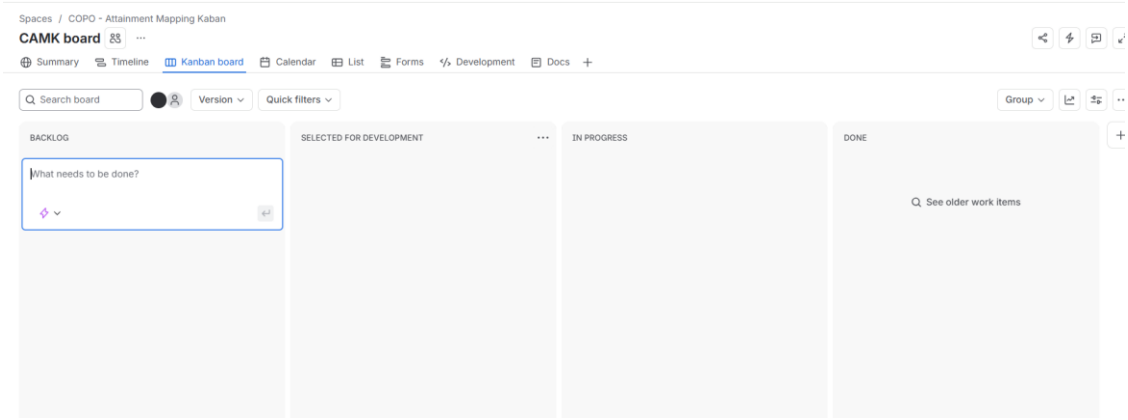
6C:


Complete COPO Sprint 3
This sprint contains 1 completed work item.
That's all of them - well done!
Cancel
Complete sprint

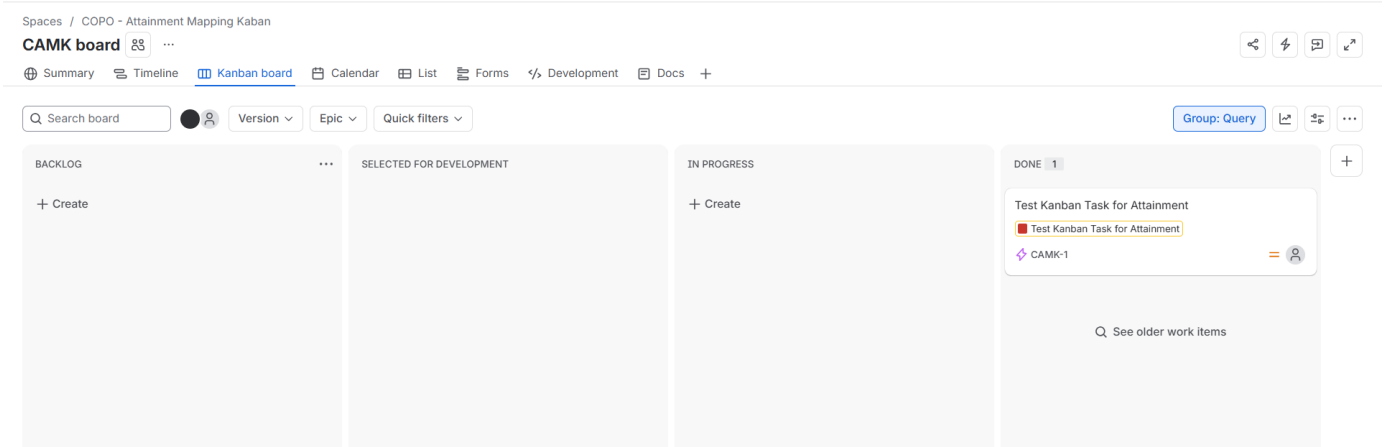
7A:


← Back to templates
Name your space
Required fields are marked with an asterisk *
Name *
COPO - Attainment Mapping Kaban
How your space is managed *
Company-managed
Learn more about space types
Share settings with an existing space
Key *
CAMK
Template
See details
Kanban
Jira
Visualise and advance your project forward using work items on a powerful board.
Step 1 of 3
Next

7B:

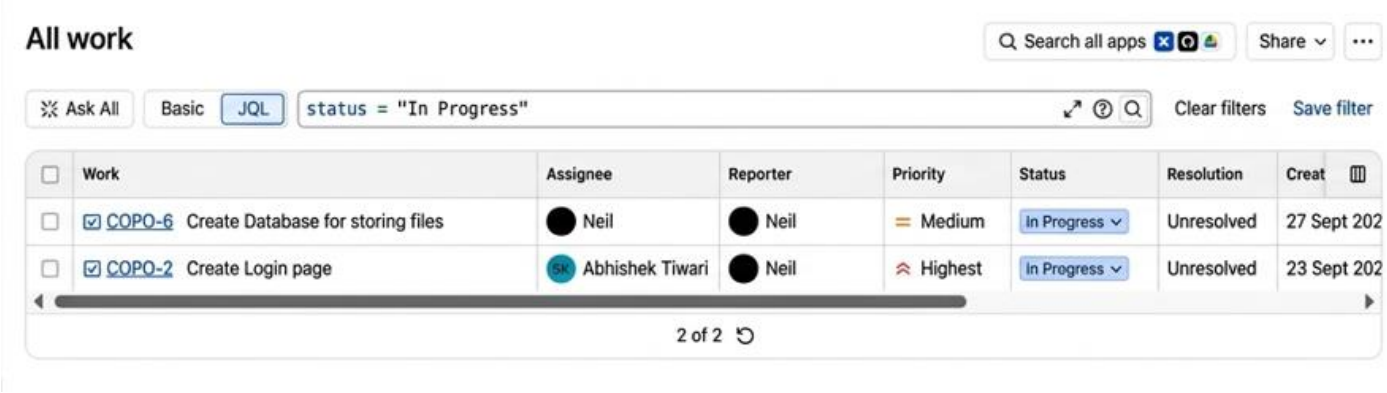

Spaces / COPO - Attainment Mapping Kaban
CAMK board
Summary
Timeline
Kanban board
Calendar
List
Forms
Development
Docs
Search board
Version
Quick filters
Group
BACKLOG
SELECTED FOR DEVELOPMENT
IN PROGRESS
DONE
What needs to be done?
See older work items

7C :

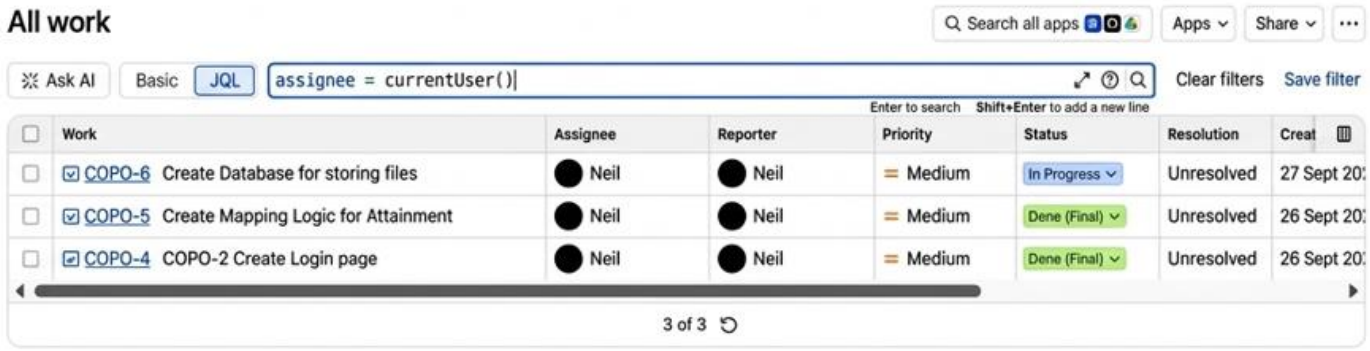

Spaces / COPO - Attainment Mapping Kaban
CAMK board
Summary
Timeline
Kanban board
Calendar
List
Forms
Development
Docs
Search board
Version
Epic
Quick filters
Group: Query
BACKLOG
SELECTED FOR DEVELOPMENT
IN PROGRESS
DONE 1
Create
Create
Test Kanban Task for Attainment
Test Kanban Task for Attainment
CAMK-1
See older work items


Spaces / COPO - Attainment Mapping Kaban / CAMK board / Reports
Control Chart
How to read this chart
How to read this chart
Shows the cycle time for your product, version or sprint. This helps you identify whether data from the current process can be used to determine future performance.
Learn more
Visibility
See outliers and investigate their cause to reduce them in the future.
Efficiency
Decreasing rolling average indicates process improvements and increased throughput.
Predictability
Narrow standard deviation through process improvements to improve predictability of cycle time.
26/Sep/26 to 26/Sep/26 (All Time)
0 average
0 median
0 min time
0 max time
1 issues
Issue
Cluster of issues
Average
Rolling average 5 issue window
Standard deviation
ELAPSED TIME (DAYS)
26/Sep/26 6:45 AM
26/Sep/26 12:00 AM
26/Sep/26 3:00 AM
26/Sep/26 6:00 AM
26/Sep/26 9:00 AM
26/Sep/26 12:00 PM
26/Sep/26 3:00 PM
26/Sep/26 6:00 PM
26/Sep/26 9:00 PM
ISSUE TRANSITION DATE
Time scale
Refine report
Viewing options
Include non-working days in calculations

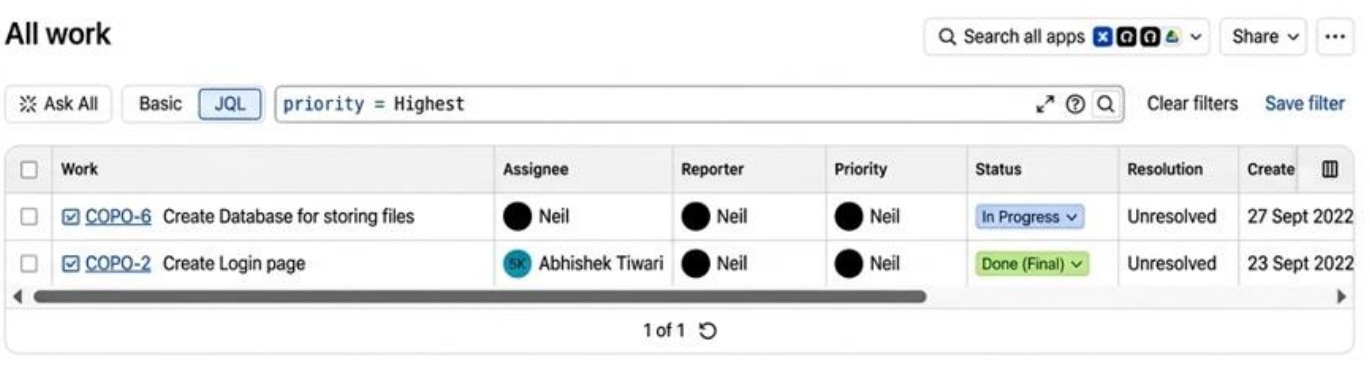
8A.1


All work
Search all apps
Share
Ask AI
Basic
JQL
status = "In Progress"
Clear filters
Save filter
Work
Assignee
Reporter
Priority
Status
Resolution
Creat
COPO-6 Create Database for storing files
Neil
Neil
Medium
In Progress
Unresolved
27 Sept 202
COPO-2 Create Login page
Abhishek Tiwari
Neil
Highest
In Progress
Unresolved
23 Sept 202
2 of 2

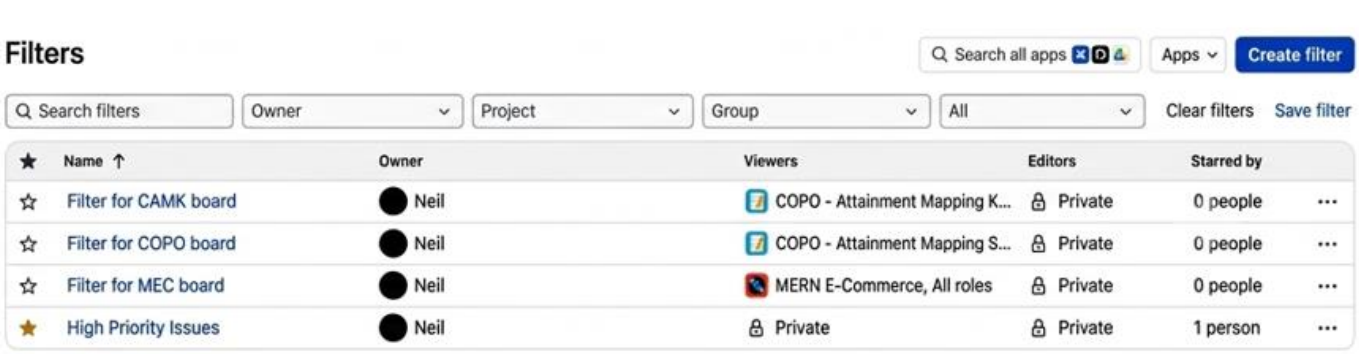
8A.2


All work
Search all apps
Apps
Share
Ask AI
Basic
JQL
assignee = currentUser()
Clear filters
Save filter
Enter to search
Shift+Enter to add a new line
Work
Assignee
Reporter
Priority
Status
Resolution
Creat
COPO-6 Create Database for storing files
Neil
Neil
Medium
In Progress
Unresolved
27 Sept 20
COPO-5 Create Mapping Logic for Attainment
Neil
Neil
Medium
Done (Final)
Unresolved
26 Sept 20
COPO-4 COPO-2 Create Login page
Neil
Neil
Medium
Done (Final)
Unresolved
26 Sept 20
3 of 3

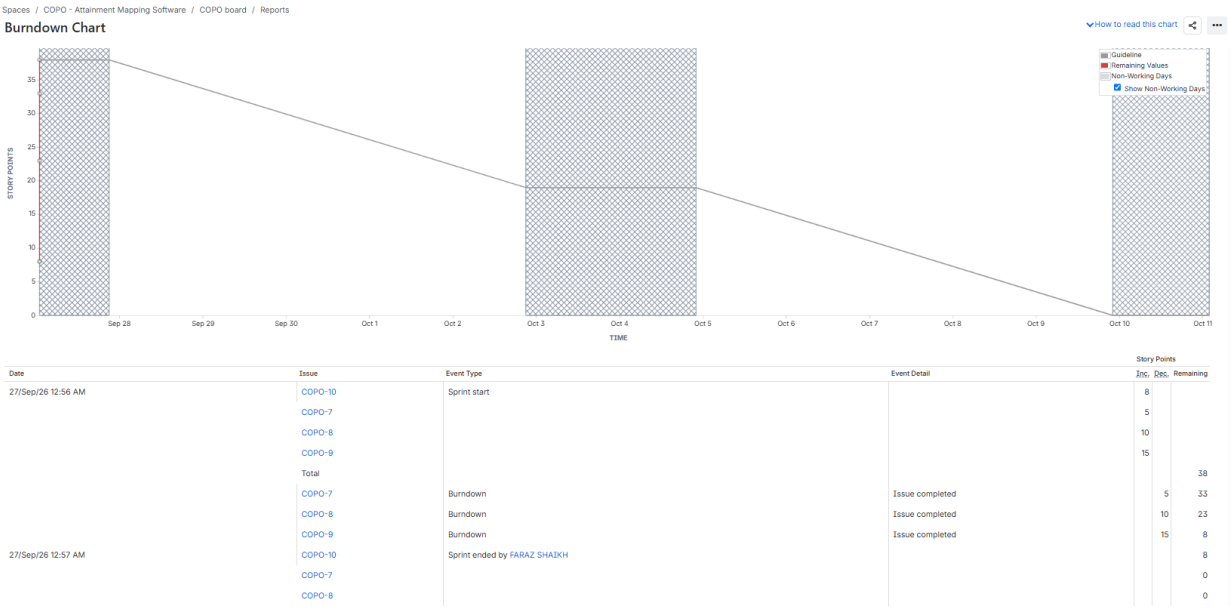
8A.3


All work
Search all apps
Share
Ask AI
Basic
JQL
priority = Highest
Clear filters
Save filter
Work
Assignee
Reporter
Priority
Status
Resolution
Create
COPO-6 Create Database for storing files
Neil
Neil
Neil
In Progress
Unresolved
27 Sept 2022
COPO-2 Create Login page
Abhishek Tiwari
Neil
Neil
Done (Final)
Unresolved
23 Sept 2022
1 of 1

8B



9A.1



Date	Issue	Event Type	Event Detail	Story Points Inc.	Dec.	Remaining
27/Sep/26 12:56 AM	COPO-10	Sprint start		8		
	COPO-7			5		
	COPO-8			10		
	COPO-9			15		
	Total					38
	COPO-7	Burndown	Issue completed		5	33
	COPO-8	Burndown	Issue completed		10	23
	COPO-9	Burndown	Issue completed		15	8
27/Sep/26 12:57 AM	COPO-10	Sprint ended by FARAZ SHAIKH				8
	COPO-7					0
	COPO-8					0

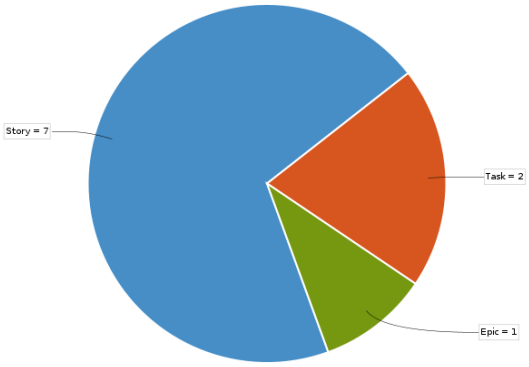
9A.2



Sprint	Commitment	Completed
COPO Sprint 2	0	0
COPO Sprint 3	0	0
COPO Sprint 4	0	0
COPO Sprint 5	38	30

9B

Chart



Data Table

	Issues	%
Story	7	70%
Task	2	20%
Epic	1	10%

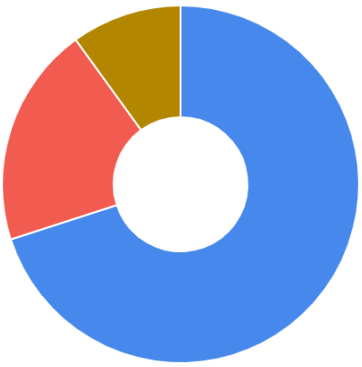
9C:

COPO Project Dashboard

Add gadget | Change layout | Done

You are currently editing your dashboard. Changes will be saved automatically.

Pie Chart: COPO - Attainment Mapping Software



Issue Type
Total Issues: **10**

Story	7
Task	2
Epic	1

Issue Statistics: COPO - Attainment Mapping Software (Assignee)

Assignee	Count	Percentage
Sufiyaan Khan	2	50%
Unassigned	1	25%
FARAZ SHAIKH	1	25%
Total	4	

Last refreshed 2 minutes ago

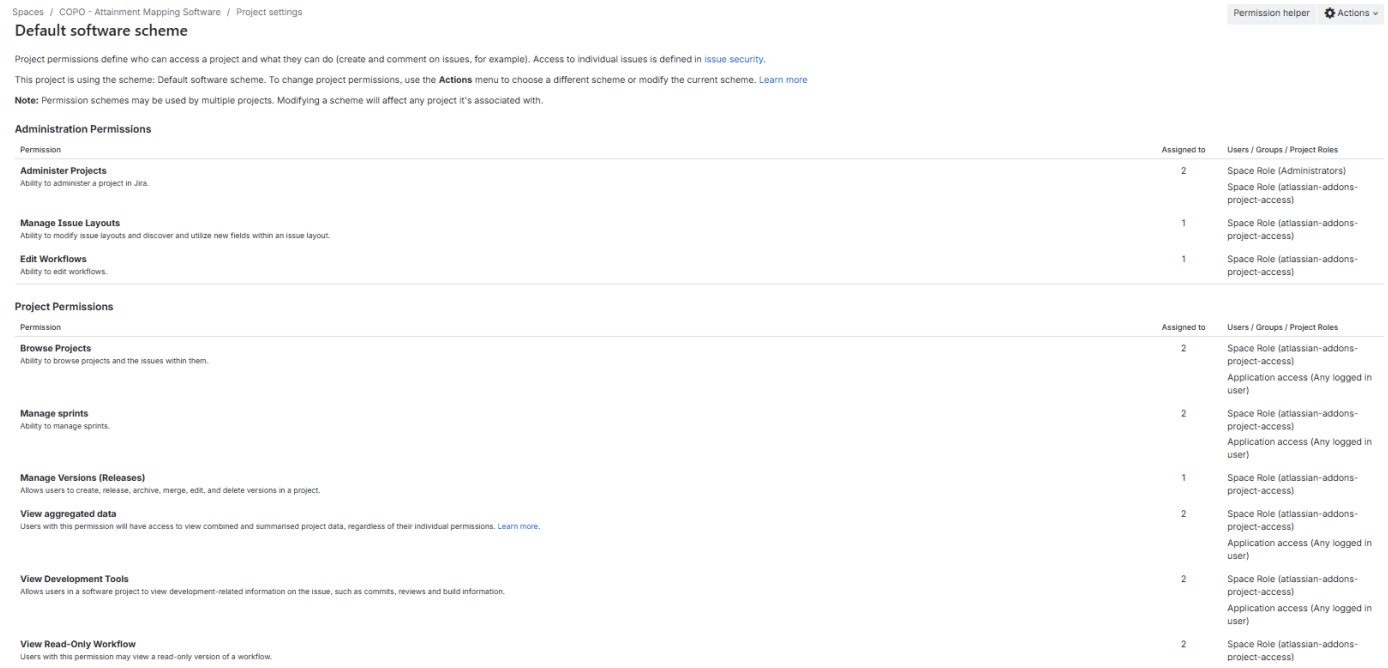
Filter Results: High Priority Issues

T	Key	Summary	P
☑	COPO-2	Create Login page	⏫

1–1 of 1

Last refreshed just now

10A.1



Spaces / COPO - Attainment Mapping Software / Project settings

Permission helper | Actions

Default software scheme

Project permissions define who can access a project and what they can do (create and comment on issues, for example). Access to individual issues is defined in issue security.

This project is using the scheme: Default software scheme. To change project permissions, use the **Actions** menu to choose a different scheme or modify the current scheme. Learn more

Note: Permission schemes may be used by multiple projects. Modifying a scheme will affect any project it's associated with.

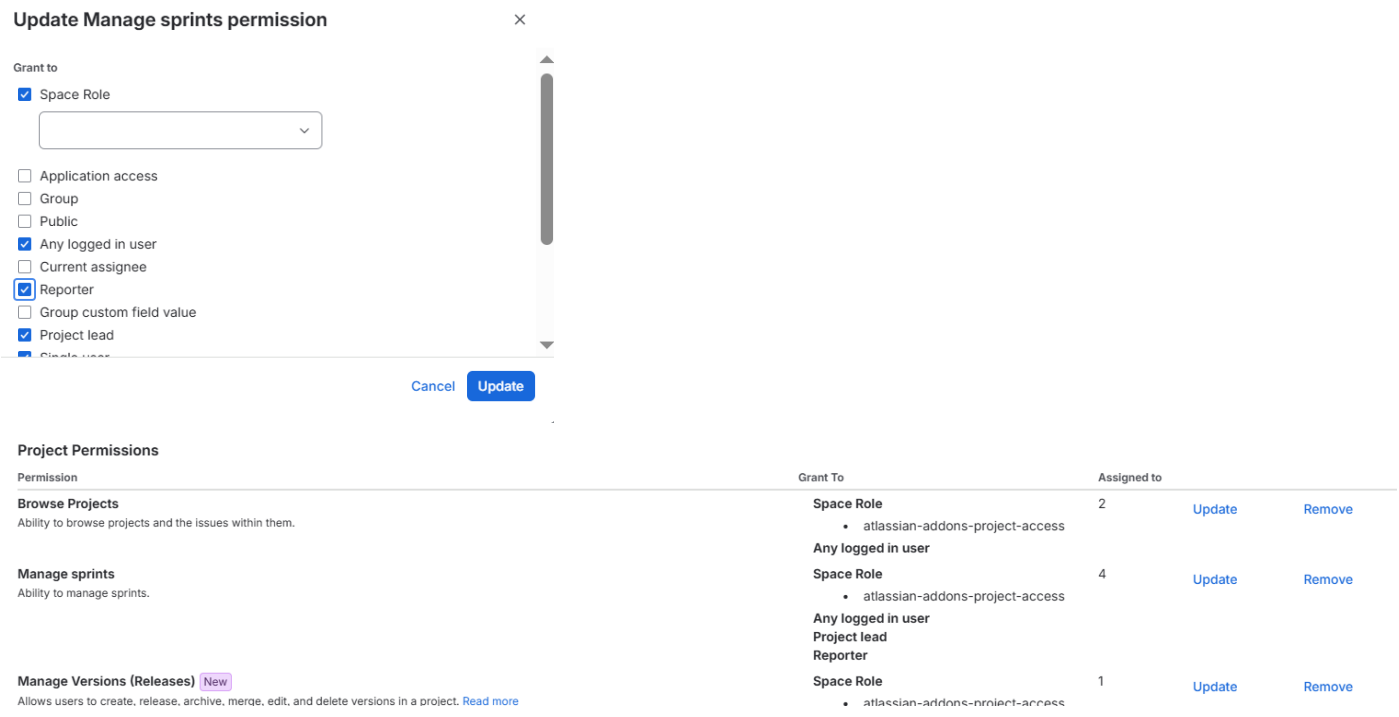
Administration Permissions

Permission	Assigned to	Users / Groups / Project Roles
Administer Projects Ability to administer a project in Jira.	2	Space Role (Administrators) Space Role (atlassian-addons-project-access)
Manage Issue Layouts Ability to modify issue layouts and discover and utilize new fields within an issue layout.	1	Space Role (atlassian-addons-project-access)
Edit Workflows Ability to edit workflows.	1	Space Role (atlassian-addons-project-access)

Project Permissions

Permission	Assigned to	Users / Groups / Project Roles
Browse Projects Ability to browse projects and the issues within them.	2	Space Role (atlassian-addons-project-access) Application access (Any logged in user)
Manage sprints Ability to manage sprints.	2	Space Role (atlassian-addons-project-access) Application access (Any logged in user)
Manage Versions (Releases) Allows users to create, release, archive, merge, edit, and delete versions in a project.	1	Space Role (atlassian-addons-project-access)
View aggregated data Users with this permission will have access to view combined and summarised project data, regardless of their individual permissions. Learn more.	2	Space Role (atlassian-addons-project-access) Application access (Any logged in user)
View Development Tools Allows users in a software project to view development-related information on the issue, such as commits, reviews and build information.	2	Space Role (atlassian-addons-project-access) Application access (Any logged in user)
View Read-Only Workflow Users with this permission may view a read-only version of a workflow.	2	Space Role (atlassian-addons-project-access)

10 A.2



Update Manage sprints permission

Grant to

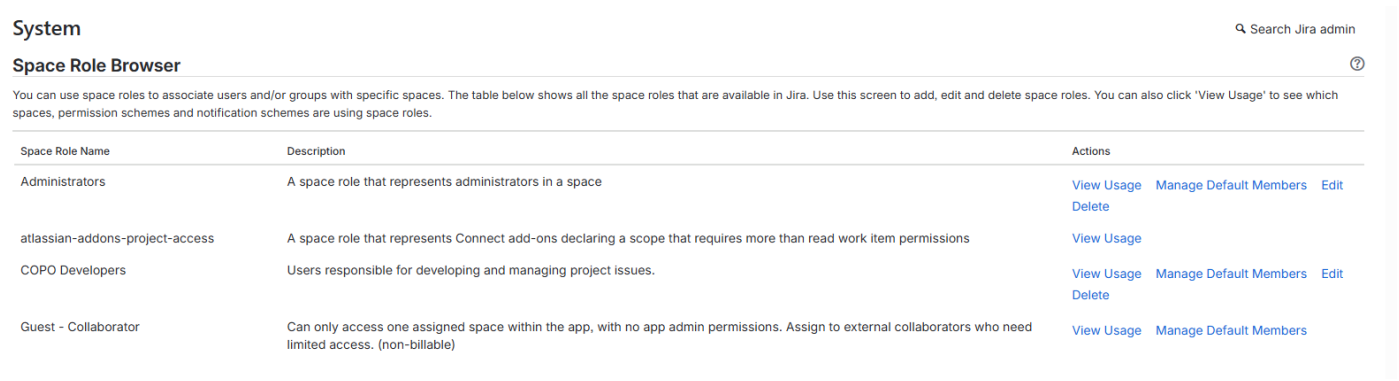
- [x] Space Role
- [ ] Application access
- [ ] Group
- [ ] Public
- [x] Any logged in user
- [ ] Current assignee
- [x] Reporter
- [ ] Group custom field value
- [x] Project lead

Cancel | Update

Project Permissions

Permission	Grant To	Assigned to		
Browse Projects Ability to browse projects and the issues within them.	**Space Role** • atlassian-addons-project-access **Any logged in user**	2	Update	Remove
Manage sprints Ability to manage sprints.	**Space Role** • atlassian-addons-project-access **Any logged in user** **Project lead** **Reporter**	4	Update	Remove
Manage Versions (Releases) New Allows users to create, release, archive, merge, edit, and delete versions in a project. Read more	**Space Role** • atlassian-addons-project-access	1	Update	Remove

10B



System

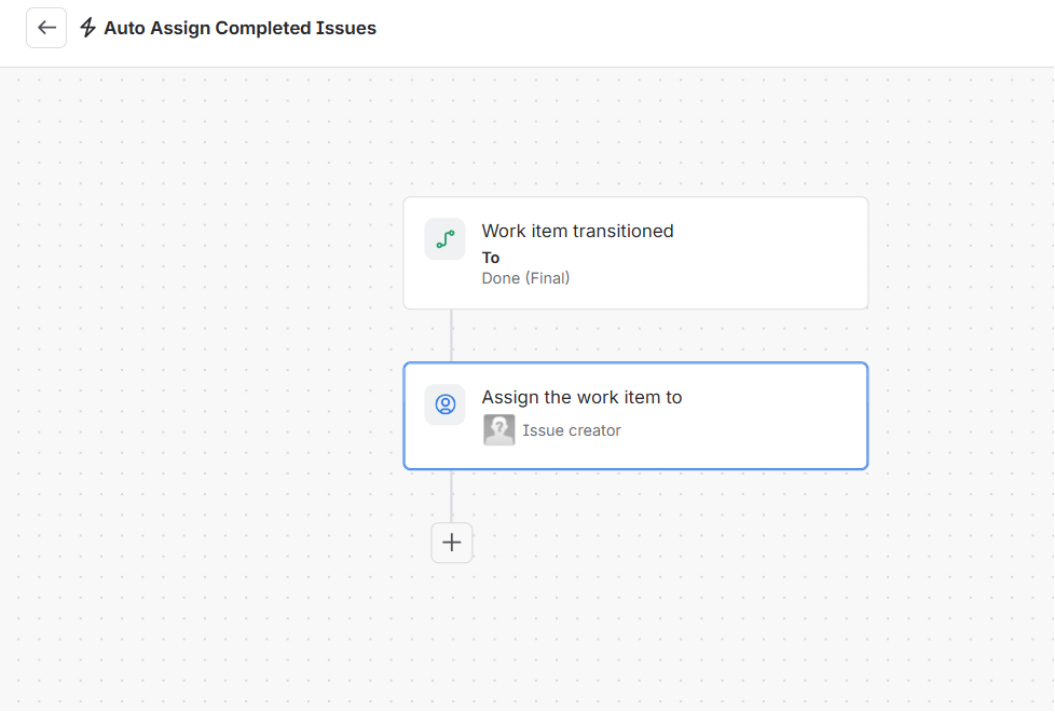
Search Jira admin

Space Role Browser

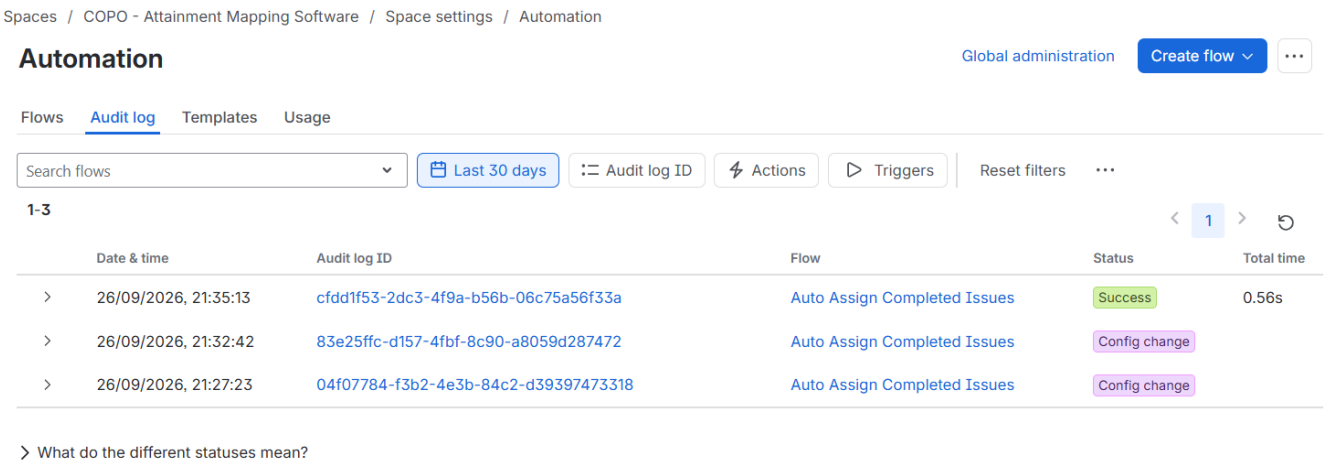
You can use space roles to associate users and/or groups with specific spaces. The table below shows all the space roles that are available in Jira. Use this screen to add, edit and delete space roles. You can also click 'View Usage' to see which spaces, permission schemes and notification schemes are using space roles.

Space Role Name	Description	Actions
Administrators	A space role that represents administrators in a space	View Usage Manage Default Members Edit Delete
atlassian-addons-project-access	A space role that represents Connect add-ons declaring a scope that requires more than read work item permissions	View Usage
COPO Developers	Users responsible for developing and managing project issues.	View Usage Manage Default Members Edit Delete
Guest - Collaborator	Can only access one assigned space within the app, with no app admin permissions. Assign to external collaborators who need limited access. (non-billable)	View Usage Manage Default Members

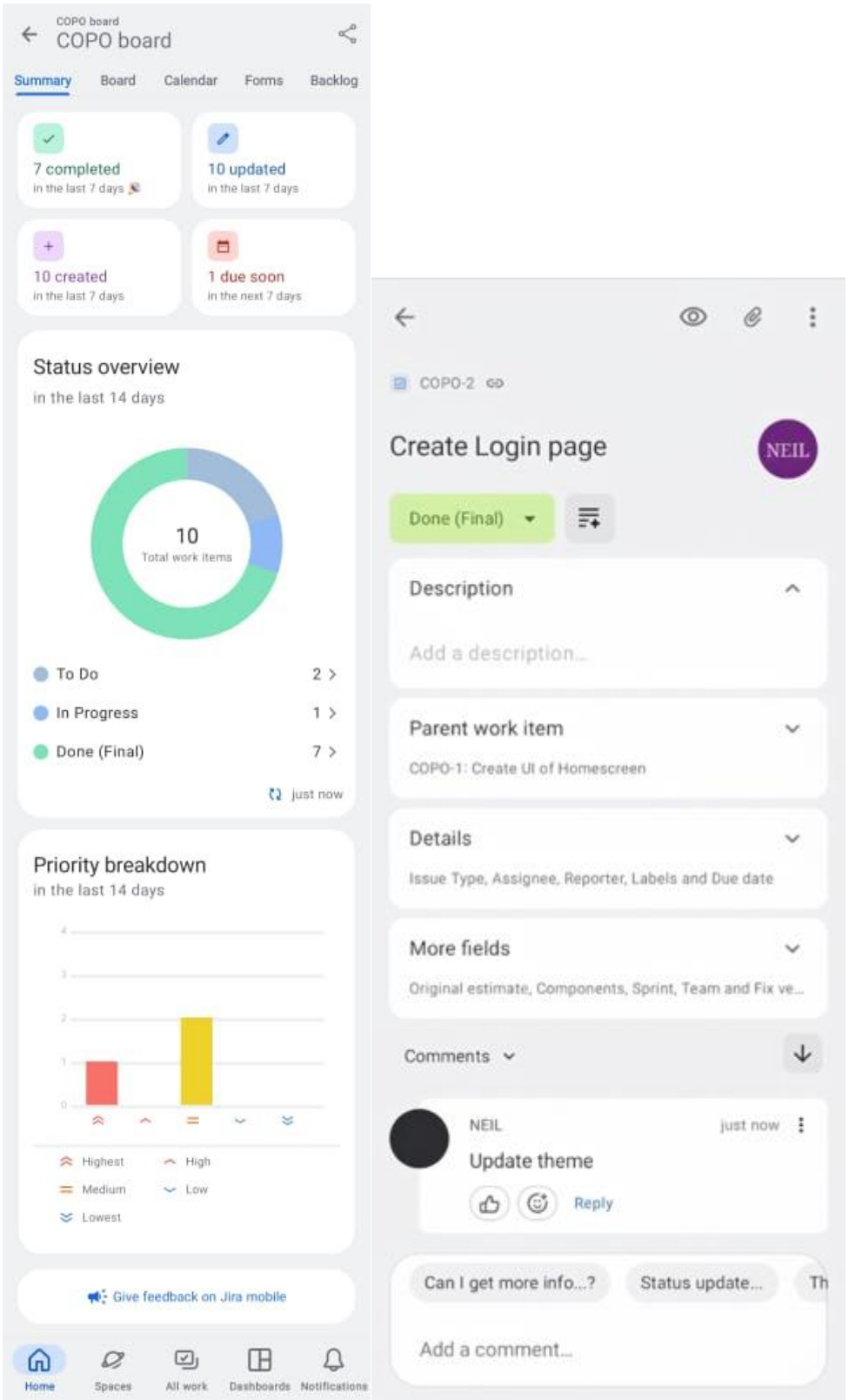
11 A and B

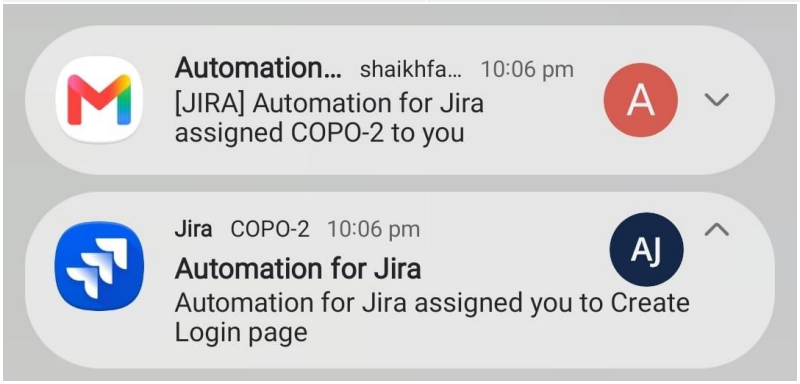

Auto Assign Completed Issues
Work item transitioned
To
Done (Final)
Assign the work item to
Issue creator

11C


Spaces / COPO - Attainment Mapping Software / Space settings / Automation
Automation
Global administration
Create flow
Flows
Audit log
Templates
Usage
Search flows
Last 30 days
Audit log ID
Actions
Triggers
Reset filters
1-3
1
Date & time
Audit log ID
Flow
Status
Total time
26/09/2026, 21:35:13
cfdd1f53-2dc3-4f9a-b56b-06c75a56f33a
Auto Assign Completed Issues
Success
0.56s
26/09/2026, 21:32:42
83e25ffc-d157-4fbf-8c90-a8059d287472
Auto Assign Completed Issues
Config change
26/09/2026, 21:27:23
04f07784-f3b2-4e3b-84c2-d39397473318
Auto Assign Completed Issues
Config change
What do the different statuses mean?

12A,B,C


COPO board
COPO board
Summary
Board
Calendar
Forms
Backlog
7 completed
in the last 7 days
10 updated
in the last 7 days
10 created
in the last 7 days
1 due soon
in the next 7 days
Status overview
in the last 14 days
10
Total work items
To Do 2
In Progress 1
Done (Final) 7
just now
Priority breakdown
in the last 14 days
Highest
High
Medium
Low
Lowest
Give feedback on Jira mobile
Home
Spaces
All work
Dashboards
Notifications
COPO-2
Create Login page
NEIL
Done (Final)
Description
Add a description...
Parent work item
COPO-1: Create UI of Homescreen
Details
Issue Type, Assignee, Reporter, Labels and Due date
More fields
Original estimate, Components, Sprint, Team and Fix ve...
Comments
NEIL
just now
Update theme
Reply
Can I get more info...?
Status update...
Add a comment...


Automation...
shaikhfa...
10:06 pm
[JIRA] Automation for Jira assigned COPO-2 to you
A
Jira
COPO-2
10:06 pm
Automation for Jira
Automation for Jira assigned you to Create Login page
AJ